SIMATS Engineering
Department of Information Technology
CSA47 - DEEP LEARNING
ASSIGNMENT 1
CO1 – LEARNING ALGORITHMS, ESTIMATION AND NEURAL NETWORK FOUNDATIONS

Nature: Individual
Duration: 1 Week **Total Marks:**50

Name: D. CHAITANYA KUMAR REDDY
Register No: 192411094

CO1: Apply and analyze learning algorithms, maximum likelihood estimation, the process of building a machine learning algorithm, multilayer perceptrons, back-propagation with stochastic gradient descent and its variants, and the curse of dimensionality to make justified modelling decisions.

Topics: Learning Algorithms, Maximum Likelihood Estimation, Building a Machine Learning Algorithm, Neural Networks – Multilayer Perceptron, Back-propagation Algorithm and its Variants (Stochastic Gradient Descent), Curse of Dimensionality.

Purpose

Investigate how learning principles, estimation theory and optimization behaviour influence the reliability of neural-network-based systems. Students must connect equations, assumptions and numerical behaviour to practical model decisions.

Scenario: Smart Manufacturing Quality Inspection

A factory plans to build an intelligent system that predicts whether a manufactured component is defective, using sensor readings such as temperature, vibration, pressure and humidity recorded during production. The factory holds labelled inspection records, and it must decide which learning approach, estimation method, network design and training procedure to use. You are required to analyze the data, select suitable techniques and justify every decision using mathematical reasoning.

TASK 1 – Learning Algorithms: Task, Experience and Performance | 4 Marks

The factory has 5,000 past inspection records. Each record contains four sensor readings and a label stating whether the component was "defective" or "acceptable". The system must label new components automatically.

a) Define the learning problem using Task (T), Experience (E) and Performance measure (P).

Ans: Task (T): classify a newly manufactured component as "defective" or "acceptable" from its four sensor readings (temperature, vibration, pressure, humidity), i.e. learn a mapping f: $\mathbb{R}^4 \rightarrow$ {defective, acceptable}.

Experience (E): the 5,000 labelled past inspection records $\{(x^{(i)}, y^{(i)})\}$, where $x^{(i)}$ is the 4-dimensional sensor vector and $y^{(i)}$ the inspector's label.

Performance (P): how well the predicted labels match the true labels on unseen components, measured with precision, recall and F1-score for the defective class (and the cost of misclassification), not just overall accuracy.

b) Identify the learning paradigm used and justify why a fixed hand-written rule is less suitable than a learning algorithm.

Ans: Paradigm: supervised learning (binary classification), because every training example comes with a ground-truth label.

A fixed hand-written rule (e.g. "defective if temperature > 80 °C") is less suitable because: (i) the defect depends on non-linear **interactions** between four sensors that are hard to guess by hand; (ii) thresholds must be set manually and cannot be tuned objectively; (iii) rules do not adapt when machines wear, materials change or sensors drift, whereas a learning algorithm can simply be retrained; (iv) rules become unmanageable as more sensors are added; (v) a learned model gives a calibrated defect probability and its error can be measured and optimised on data.

c) Explain why accuracy alone may be misleading if only 3% of components are defective, and name two better measures.

Ans: With only 3% defective components, a trivial classifier that always predicts "acceptable" achieves **97% accuracy** while catching **0 defects** (recall = 0). Accuracy is dominated by the majority class and hides the failure on the minority class we actually care about.

Better measures: **(1) Recall** = TP / (TP + FN) – the fraction of real defects that are caught; **(2) Precision** = TP / (TP + FP) – the fraction of flagged components that are really defective. Their harmonic mean, **F1 = 2PR / (P + R)**, (or PR-AUC) summarises both.

Evidence Expected: *T, E, P definition, identification of the learning paradigm, justification, and metric reasoning.*

TASK 2 – Capacity, Overfitting and Generalization | 3 Marks

Three candidate models are evaluated. Model X has a training error of 18% and a validation error of 20%. Model Y has a training error of 2% and a validation error of 19%. Model Z has a training error of 6% and a validation error of 8%.

a) Identify which model underfits, which overfits and which generalizes best.

Ans: Model X – underfits (training 18% and validation 20% are both high, gap only 2%: high bias). **Model Y – overfits** (training 2% but validation 19%: gap = 17%, it memorises the training data: high variance). **Model Z – generalizes best** (low validation error 8% and small gap of 2%).

b) Explain how model capacity and the generalization gap are related.

Ans: Capacity is the range of functions a model can fit (number of parameters, depth, polynomial degree). Generalization gap = validation error − training error.

As capacity increases, training error keeps falling, while validation error first falls and then rises (U-shaped curve). **Low capacity** → underfitting: both errors high, gap small (X). **Too much capacity** relative to the amount of data → the model fits noise, training error is tiny and the gap becomes large (Y). The best model has capacity matched to the data, where validation error is minimal and the gap is small (Z).

c) Suggest one suitable improvement each for Models X and Y.

Ans: Model X (underfit): increase capacity – add hidden units/layers or non-linear/polynomial features, reduce regularization, or train longer.

Model Y (overfit): add regularization (L2 weight decay or dropout), use early stopping, collect more training data / augment, or reduce model size.

Evidence Expected: *Correct identification, capacity explanation, and justified remedies.*

TASK 3 – Maximum Likelihood Estimation: Bernoulli Model | 4 Marks

In one production batch, 50 components are inspected and 7 are found defective. Let each component be defective with unknown probability p, independently of the others.

a) Write the likelihood and log-likelihood function for the observed data.

Ans: Observed data: n = 50, k = 7 defective, with each component Bernoulli(p) and independent.

Likelihood: $L(p) = \prod p^{x_i} (1 - p)^{1-x_i} = p^7 (1 - p)^{43}$ (a constant binomial coefficient C(50,7) can be added; it does not affect the maximiser).

Log-likelihood: $\ell(p) = 7 \ln p + 43 \ln(1 - p)$.

b) Derive the maximum likelihood estimate of p and compute its value.

Ans: Set the derivative to zero: $d\ell/dp = 7/p - 43/(1 - p) = 0$

$\Rightarrow 7(1 - p) = 43p \Rightarrow 7 = 50p \Rightarrow \mathbf{\hat{p} = 7/50 = 0.14}$.

Second derivative: $d^2\ell/dp^2 = -7/p^2 - 43/(1 - p)^2 < 0$, so this is a maximum. In general $\hat{p} = k/n$ (the sample proportion).

c) Using your estimate, calculate the probability that the next three inspected components are all acceptable, and comment on the reliability of the estimate for a small sample.

Ans: P(acceptable) = $1 - \hat{p} = 0.86$, so P(next three all acceptable) = $0.86^3 = \mathbf{0.636}$ ($\approx 63.6\%$).

Reliability: the standard error is $\sqrt{(\hat{p}(1 - \hat{p})/n)} = \sqrt{(0.14 \times 0.86 / 50)} \approx 0.049$, giving an approximate 95% interval of $0.14 \pm 0.096 \approx (0.044, 0.236)$. The true p could plausibly be anywhere from ~4% to ~24%, so the three-acceptable probability could range from about 0.45 to 0.87. With only 50 samples the MLE is therefore imprecise (variance $p(1 - p)/n$ shrinks only as 1/n); a larger sample, or a Bayesian prior, is needed for dependable estimates.

Evidence Expected: *Likelihood expression, derivation steps, numerical results and interpretation.*

TASK 4 – Maximum Likelihood Estimation: Gaussian Model and Link to MSE | 5 Marks

A furnace temperature sensor records the readings 68, 70, 72, 74 and 76 °C, assumed to be drawn independently from a Gaussian distribution $N(\mu, \sigma^2)$.

a) Write the Gaussian likelihood and log-likelihood function.

Ans: Likelihood: $L(\mu, \sigma^2) = \prod_i (2\pi\sigma^2)^{(-1/2)} \exp[-(x_i - \mu)^2 / (2\sigma^2)] = (2\pi\sigma^2)^{(-n/2)} \exp[-\Sigma(x_i - \mu)^2 / (2\sigma^2)]$.

Log-likelihood: $\ell(\mu, \sigma^2) = -(n/2) \ln(2\pi) - (n/2) \ln \sigma^2 - (1/(2\sigma^2)) \Sigma(x_i - \mu)^2$.

b) Derive the MLE of μ and σ^2 and compute their values for the given readings.

Ans: For μ: $\partial\ell/\partial\mu = (1/\sigma^2) \Sigma(x_i - \mu) = 0 \Rightarrow \mathbf{\hat{\mu} = (1/n) \Sigma x_i}$.

For σ^2: $\partial\ell/\partial\sigma^2 = -n/(2\sigma^2) + \Sigma(x_i - \mu)^2/(2\sigma^4) = 0 \Rightarrow \mathbf{\hat{\sigma}^2 = (1/n) \Sigma(x_i - \hat{\mu})^2}$.

Numbers: $\Sigma x_i = 68 + 70 + 72 + 74 + 76 = 360$, so $\hat{\mu} = 360/5 = \mathbf{72\ °C}$. Deviations: −4, −2, 0, 2, 4 → squares 16, 4, 0, 4, 16, sum = 40, so $\hat{\sigma}^2 = 40/5 = \mathbf{8\ °C^2}$ ($\hat{\sigma} \approx 2.83$ °C).

c) Explain why the MLE of σ^2 is biased, compute the unbiased estimate using n − 1, and compare the two values.

Ans: Why biased: $\hat{\mu}$ is itself computed from the same data, so the sample points lie closer to $\bar{x}$ than to the true μ: $\Sigma(x_i - \bar{x})^2 = \Sigma(x_i - \mu)^2 - n(\bar{x} - \mu)^2$. Taking expectations gives $E[\hat{\sigma}^2] = ((n - 1)/n) \sigma^2 < \sigma^2$, so the MLE systematically underestimates the variance (one degree of freedom is used up by estimating μ).

Unbiased estimate: $s^2 = (1/(n - 1)) \Sigma(x_i - \bar{x})^2 = 40/4 = \mathbf{10\ °C^2}$.

Comparison: MLE = 8 versus unbiased = 10; the ratio is 8/10 = (n − 1)/n = 4/5, i.e. the MLE is 20% too small for n = 5. The gap vanishes as n grows (the MLE is still consistent).

d) For the regression model $y = wx + b + \varepsilon$ with $\varepsilon \sim N(0, \sigma^2)$, show that maximizing the likelihood is equivalent to minimizing the mean squared error.

Ans: The model says $y \mid x \sim N(wx + b, \sigma^2)$, so $p(y_i \mid x_i) = (2\pi\sigma^2)^{(-1/2)} \exp[-(y_i - wx_i - b)^2 / (2\sigma^2)]$.

Log-likelihood of m independent samples: $\ell(w, b) = -(m/2) \ln(2\pi\sigma^2) - (1/(2\sigma^2)) \Sigma(y_i - wx_i - b)^2$.

The first term does not depend on w or b, and $1/(2\sigma^2) > 0$ is a positive constant. Hence argmax $\ell(w, b)$ = argmin $\Sigma(y_i - (wx_i + b))^2$ = argmin $(1/m)\ \Sigma(y_i - \hat{y}_i)^2$ = **argmin MSE**. So maximum-likelihood estimation under Gaussian noise is exactly least-squares (MSE) regression.

Evidence Expected: *Derivations, numerical solutions, bias discussion and the MLE–MSE equivalence.*

TASK 5 – Building a Machine Learning Algorithm | 5 Marks

The factory wants to predict tool wear y (in units of 0.1 mm) from running time x (in hours). Four observations are available: (x, y) = (1, 3), (2, 5), (3, 6), (4, 9). The proposed model is $\hat{y} = wx + b$ with cost $J = (1/m)\ \Sigma\ (\hat{y} - y)^2$.

a) Identify the dataset, model, cost function and optimization procedure that together form this learning algorithm.

Ans: Dataset: D = {(1,3), (2,5), (3,6), (4,9)}, m = 4 (x = running time in hours, y = tool wear in 0.1 mm).

Model: linear regression $\hat{y} = wx + b$ with parameters w and b.

Cost function: $J(w, b) = (1/m)\ \Sigma(\hat{y}_i - y_i)^2$ (mean squared error).

Optimization procedure: gradient descent, $w \leftarrow w - \alpha\ \partial J/\partial w$, $b \leftarrow b - \alpha\ \partial J/\partial b$.

b) Derive the expressions for $\partial J/\partial w$ and $\partial J/\partial b$.

Ans: $J = (1/m)\ \Sigma(wx_i + b - y_i)^2$. Applying the chain rule:

$$\partial J/\partial w = (2/m)\ \Sigma\ (\hat{y}_i - y_i)\ x_i \qquad \partial J/\partial b = (2/m)\ \Sigma\ (\hat{y}_i - y_i)$$

c) Starting from w = 0 and b = 0 with learning rate $\alpha = 0.05$, compute the gradients and perform one gradient descent update. Report the new w and b.

Ans: At w = 0, b = 0: $\hat{y} = 0$ for all points, so errors $(\hat{y} - y) = -3, -5, -6, -9$.

$\Sigma(\hat{y} - y) = -23$; $\Sigma(\hat{y} - y)x = (-3)(1) + (-5)(2) + (-6)(3) + (-9)(4) = -3 - 10 - 18 - 36 = -67$.

$\partial J/\partial w = (2/4)(-67) = \mathbf{-33.5}$; $\partial J/\partial b = (2/4)(-23) = \mathbf{-11.5}$.

w_new = 0 − 0.05(−33.5) = **1.675**; b_new = 0 − 0.05(−11.5) = **0.575**.

Check: J falls from 37.75 (at w = b = 0) to ≈ 1.21 after this one update (new predictions 2.25, 3.925, 5.6, 7.275).

d) Differentiate between parameters and hyperparameters in this problem, and explain the effect of choosing α too large or too small.

Ans: Parameters (learned from data by gradient descent): w and b. **Hyperparameters** (set by the designer, not learned by the update rule): the learning rate α, number of iterations/epochs, initial values of w and b, the choice of linear model and the cost function.

α too large: the update overshoots the minimum, the cost oscillates or diverges. For this problem the Hessian of J has largest eigenvalue ≈ 16.7, so gradient descent is stable only for $\alpha < 2/16.7 \approx$

0.12; α = 0.05 is safe. **α too small:** progress is very slow (the other eigenvalue is only ≈ 0.30), needing many iterations and more computation to converge.

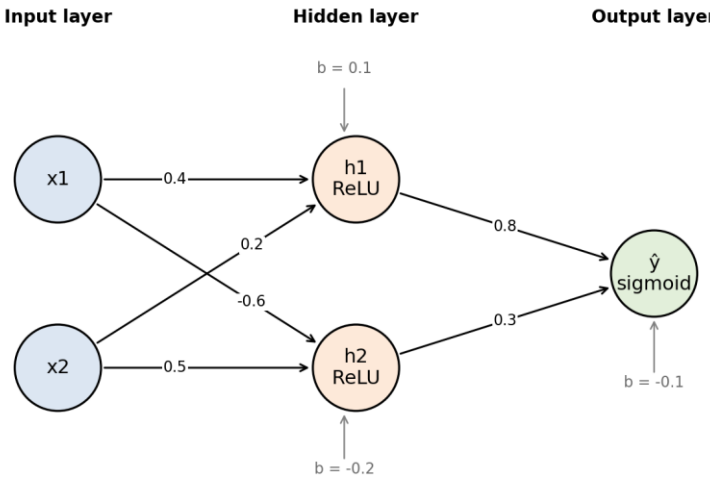
Evidence Expected: *Four-component specification, gradient derivation, numerical update and hyperparameter reasoning.*

TASK 6 – Multilayer Perceptron: Architecture and Forward Pass | 4 Marks

A Multilayer Perceptron has 2 inputs, one hidden layer with 2 ReLU neurons, and 1 sigmoid output neuron. The input is x = (0.5, 1.0). Hidden weights and biases: h_1 uses weights (0.4, 0.2) and bias 0.1; h_2 uses weights (−0.6, 0.5) and bias −0.2. Output weights are $v_1 = 0.8$ and $v_2 = 0.3$ with output bias −0.1.

a) Draw the network and count the number of trainable parameters.

Ans: Network: 2 inputs → 2 hidden ReLU neurons (h_1, h_2) → 1 sigmoid output.



Trainable parameters: hidden layer = 2 × 2 weights + 2 biases = 6; output layer = 2 weights + 1 bias = 3. **Total = 9 parameters.**

b) Perform the forward pass and compute h_1, h_2 and the output ŷ.

Ans: $z_1 = 0.4(0.5) + 0.2(1.0) + 0.1 = 0.5 \Rightarrow$ **$h_1 = \text{ReLU}(0.5) = 0.5$**

$z_2 = -0.6(0.5) + 0.5(1.0) - 0.2 = 0.0 \Rightarrow$ **$h_2 = \text{ReLU}(0) = 0$**

z_out = $0.8(0.5) + 0.3(0) - 0.1 = 0.3 \Rightarrow$ **$\hat{y} = \sigma(0.3) = 1/(1 + e^{(-0.3)}) \approx 0.5744$**

c) Explain why a hidden layer with a non-linear activation is necessary, and what happens if all activations are linear.

Ans: A composition of linear maps is itself linear. If all activations are linear, then $\hat{y} = V(Wx + b) + c = (VW)x + (Vb + c)$, which is just a single linear layer no matter how many layers are stacked; the network could only learn linear decision boundaries and cannot solve problems such as XOR or sensor interactions.

A non-linear activation (ReLU) in the hidden layer lets each neuron bend the function, so the network can build piecewise-linear/curved boundaries and approximate any continuous function (universal approximation). This is necessary for defect patterns that depend on combinations of temperature, vibration, pressure and humidity.

Evidence Expected: *Network diagram, parameter count, forward-pass calculations and explanation of non-linearity.*

TASK 7 – Back-propagation Algorithm | 5 Marks

Use the network and input of Task 6 with target $t = 1$ and loss $L = \frac{1}{2}(t - \hat{y})^2$. The output neuron is sigmoid and the hidden neurons are ReLU.

a) Write the chain-rule expressions for $\partial L/\partial v_1$ and $\partial L/\partial w_{11}$, where w_{11} is the weight from the first input to h_1.

Ans: Let $z_o = v_1h_1 + v_2h_2 + c$, $\hat{y} = \sigma(z_o)$, $L = \frac{1}{2}(t - \hat{y})^2$, and $z_1 = w_{11}x_1 + w_{12}x_2 + b_1$, $h_1 = \text{ReLU}(z_1)$.

$\boldsymbol{\partial L/\partial v_1 = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial z_o)(\partial z_o/\partial v_1) = (\hat{y} - t) \cdot \hat{y}(1 - \hat{y}) \cdot h_1}$

$\boldsymbol{\partial L/\partial w_{11} = (\partial L/\partial \hat{y})(\partial \hat{y}/\partial z_o)(\partial z_o/\partial h_1)(\partial h_1/\partial z_1)(\partial z_1/\partial w_{11}) = (\hat{y} - t) \cdot \hat{y}(1 - \hat{y}) \cdot v_1 \cdot \text{ReLU}'(z_1) \cdot x_1}$

b) Compute $\partial L/\partial v_1$, $\partial L/\partial v_2$, $\partial L/\partial$(output bias) and $\partial L/\partial w_{11}$ numerically.

Ans: From Task 6: $\hat{y} = 0.5744$, $h_1 = 0.5$, $h_2 = 0$, $z_1 = 0.5 > 0$ (so $\text{ReLU}' = 1$), $x_1 = 0.5$.

$\hat{y} - t = 0.5744 - 1 = -0.4256$; $\hat{y}(1 - \hat{y}) = 0.5744 \times 0.4256 = 0.2445$.

Output error term: $\delta_o = (\hat{y} - t)\,\hat{y}(1 - \hat{y}) = -0.4256 \times 0.2445 \approx \mathbf{-0.1040}$.

$\partial L/\partial v_1 = \delta_o \cdot h_1 = -0.1040 \times 0.5 = \mathbf{-0.0520}$

$\partial L/\partial v_2 = \delta_o \cdot h_2 = -0.1040 \times 0 = \mathbf{0}$ (h_2 is inactive)

$\partial L/\partial$(output bias) $= \delta_o = \mathbf{-0.1040}$

$\partial L/\partial w_{11} = \delta_o \cdot v_1 \cdot 1 \cdot x_1 = -0.1040 \times 0.8 \times 1 \times 0.5 = \mathbf{-0.0416}$

c) Update v_1 and w_{11} using learning rate $\eta = 0.1$.

Ans: $v_1 \leftarrow v_1 - \eta\, \partial L/\partial v_1 = 0.8 - 0.1(-0.0520) = \mathbf{0.8052}$

$w_{11} \leftarrow w_{11} - \eta\, \partial L/\partial w_{11} = 0.4 - 0.1(-0.0416) = \mathbf{0.4042}$

d) Explain why back-propagation is more efficient than computing every derivative independently.

Ans: Back-propagation applies the chain rule from the output backwards and **reuses** shared intermediate terms (the error terms δ) for many parameters; e.g. δ_o above is used for v_1, v_2, the output bias and every hidden weight. All P gradients are obtained with about one forward and one backward pass, a cost of O(P), about the same as evaluating the network.

Computing each derivative independently (e.g. numerical differences or re-deriving the full chain for every weight) needs a separate forward pass per parameter, i.e. $O(P^2)$ work with a lot of repeated computation, which is impractical for networks with millions of weights.

Evidence Expected: *Chain-rule derivation, step-by-step numerical gradients, weight update and efficiency explanation.*

TASK 8 – Vanishing Gradient and Activation Functions | 3 Marks

The factory deepens its network to 10 hidden layers, all using sigmoid activation. Training loss barely decreases in the early layers.

a) Show that the derivative of the sigmoid is at most 0.25 and calculate the upper bound 0.25^{10} on the product of activation derivatives across 10 layers.

Ans: $\sigma(z) = 1/(1 + e^{\wedge}(-z))$ and $\sigma'(z) = \sigma(z)(1 - \sigma(z))$. Let $s = \sigma(z) \in (0, 1)$. Then $s(1 - s)$ is a downward parabola with maximum at $s = \frac{1}{2}$, giving $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$. Hence $\boldsymbol{\sigma'(z) \leq 0.25}$, attained at $z = 0$.

Across 10 layers the product of activation derivatives is at most $0.25^{10} = 1/4^{10} = 1/1{,}048{,}576 \approx \mathbf{9.54 \times 10^{-7}}$.

b) Explain how this causes vanishing gradients and what it does to learning in the early layers.

Ans: By back-propagation the gradient reaching layer ℓ contains a product of terms $W \cdot \sigma'(z)$ from all later layers. Each σ' factor is ≤ 0.25, so the gradient shrinks exponentially with depth (about 10^{-6} of its size after 10 layers unless weights are very large). This is the **vanishing gradient problem**.

Effect: the early layers receive almost zero gradient, their weights hardly change, the loss barely decreases and they fail to learn useful low-level features, while only the last few layers train. Training becomes very slow or stalls.

c) Suggest two remedies and justify each.

Ans: Remedy 1 – ReLU / Leaky ReLU activations: the derivative is 1 for positive inputs (and a small constant for Leaky ReLU), so the gradient is not multiplied by a factor ≤ 0.25 at every layer and passes through active units undiminished.

Remedy 2 – Batch normalization with He/Xavier initialization (or residual connections): these keep the activations and gradients at a stable scale in every layer, and keep pre-activations away from the saturated tails of the sigmoid, so gradients do not decay layer by layer. Residual (skip) connections add an identity path for the gradient.

Evidence Expected: *Derivative bound, numerical computation, explanation of the effect and justified remedies.*

TASK 9 – Batch, Stochastic and Mini-batch Gradient Descent | 3 Marks

The factory trains on N = 1,000,000 inspection records and compares batch gradient descent, stochastic gradient descent (SGD) and mini-batch gradient descent with a batch size of 100.

a) Calculate the number of parameter updates in one epoch for each method.

Ans: Batch GD: 1 update per epoch. **SGD (one sample per update):** N = **1,000,000** updates per epoch. **Mini-batch (B = 100):** N/B = 1,000,000/100 = **10,000** updates per epoch.

b) Compare the methods in terms of gradient noise, memory use, cost per update and convergence behaviour.

Ans: Comparison:

Aspect	Batch GD	SGD	Mini-batch (B = 100)
Gradient noise	None (exact gradient)	Very high (single sample)	Moderate (variance $\propto 1/B$)
Memory use	Highest – needs all N records per step	Lowest – one record	Low – only B records
Cost per update	O(N·P), very expensive	O(P), very cheap	O(B·P), cheap and vectorised
Convergence	Smooth and stable but only 1 update/epoch, so slow	Many updates but erratic, oscillates around the minimum unless α is decayed; poor hardware use	Fast and fairly stable; efficient on GPU; some noise helps escape saddle points

c) Recommend one method for this dataset and justify the choice.

Ans: Recommendation: mini-batch gradient descent (batch size 100, or up to a few hundred). It gives 10,000 parameter updates per epoch, so progress is far faster than batch GD; each update is cheap and fits easily in memory; matrix operations are vectorised on a GPU/CPU (unlike pure SGD); and the moderate gradient noise gives stable convergence and helps generalization. With 10^6 records, full-batch is too costly and pure SGD is too noisy and inefficient.

Evidence Expected: *Update counts, comparison of the three methods and a justified recommendation.*

TASK 10 – SGD Variants: Momentum, RMSProp and Adam | 3 Marks

A single weight starts at $w_0 = 2$. Two successive gradients are $g_1 = 4$ and $g_2 = 1$, with learning rate $\eta = 0.1$. For momentum, use $v_t = \beta v_{t-1} + g_t$ and $w_t = w_{t-1} - \eta v_t$ with $\beta = 0.9$ and $v_0 = 0$.

a) Compute w_1 and w_2 using plain SGD and then using SGD with momentum.

Ans: Plain SGD: $w_1 = 2 - 0.1(4) =$ **1.6**; $w_2 = 1.6 - 0.1(1) =$ **1.5**.

SGD with momentum ($\beta = 0.9$, $v_0 = 0$):

$v_1 = 0.9(0) + 4 = 4 \Rightarrow w_1 = 2 - 0.1(4) =$ **1.6**

$v_2 = 0.9(4) + 1 = 4.6 \Rightarrow w_2 = 1.6 - 0.1(4.6) =$ **1.14**

Although the gradient dropped from 4 to 1, momentum remembers the earlier gradient, so the second step is 0.46 instead of 0.10 ($w_2 = 1.14$ vs 1.5).

b) Explain which problem of plain SGD is addressed by momentum, by RMSProp and by Adam.

Ans: Momentum accumulates an exponentially-weighted average of past gradients (a "velocity"). It fixes the slow progress and zig-zag oscillation of plain SGD in narrow valleys / on noisy gradients: consistent directions are amplified and oscillating directions cancel.

RMSProp divides the step by a running RMS of recent gradients, $s \leftarrow \rho s + (1 - \rho)g^2$, $w \leftarrow w - \eta\, g/(\sqrt{s} + \varepsilon)$. It fixes the problem of a single global learning rate being unsuitable for parameters with very different gradient magnitudes, by giving each parameter its own adaptive step size (and it avoids AdaGrad's ever-shrinking rate).

Adam combines both: a momentum-style first moment m and an RMSProp-style second moment v, with bias correction, $w \leftarrow w - \eta\, \hat{m}/(\sqrt{\hat{v}} + \varepsilon)$. It addresses oscillation and per-parameter scaling together, and needs little tuning.

c) State one situation where Adam is preferred over plain SGD and one situation where plain SGD may still be preferred.

Ans: Adam preferred: training a deep MLP on many sensor features with different scales, noisy or sparse gradients, and when fast convergence with little tuning is wanted (e.g. the factory's 500-feature network).

Plain SGD (with a tuned learning-rate schedule) may still be preferred: when the best final generalization is needed and there is time to tune it (it often generalizes slightly better than Adam in large vision models), for simple or convex models such as logistic regression, or when memory is tight, since Adam stores two extra values per parameter.

Evidence Expected: *Numerical updates for both methods, explanation of each variant and context-based choice.*

TASK 11 – Curse of Dimensionality | 3 Marks

The factory plans to extend the sensor set from 4 features to 100 features. Every feature is scaled to [0, 1] and divided into 10 equal bins. Only 5,000 labelled records are available.

a) Calculate the number of cells in the feature space for d = 4 and d = 100, and compare it with 5,000 records.

Ans: d = 4: $10^4 =$ **10,000 cells**. **d = 100:** 10^{100} cells.

For d = 4, 5,000 records ÷ 10,000 cells = 0.5 record per cell: already sparse, with at most half the cells occupied. For d = 100, $5{,}000 / 10^{100} = 5 \times 10^{-97}$ records per cell: virtually every cell is empty and each record sits alone, so the data cannot 'cover' the space at all.

b) For uniformly distributed data, a sub-cube must contain 10% of the data. Using r = 0.1^(1/d), compute the required edge length for d = 2, 10 and 100 and interpret the result.

Ans: r = 0.1^(1/d):

d = 2: r = 0.1^0.5 = **0.316**

d = 10: r = 0.1^0.1 = **0.794**

d = 100: r = 0.1^0.01 = **0.977**

Interpretation: to capture just 10% of the data in d = 100 the sub-cube must span about 97.7% of the range of every feature, i.e. almost the whole space. So a "local" neighbourhood is no longer local; points are all far apart.

c) Explain the consequences for a k-nearest neighbour classifier and suggest two ways to reduce the problem.

Ans: Consequences for k-NN: distances between points concentrate, so the nearest and farthest neighbours are almost equally far away; the "nearest" neighbours are not truly similar and their labels are not informative, so accuracy drops. Irrelevant or correlated features dominate the distance, the data are too sparse for reliable estimates, and prediction is expensive (O(N·d) per query, all data stored).

Two ways to reduce the problem: (1) **dimensionality reduction** such as PCA (or an autoencoder) to keep the few directions with most variance; (2) **feature selection** – drop irrelevant/redundant sensors. (More labelled data and regularization also help.)

Evidence Expected: *Cell counts, edge-length calculations, interpretation and mitigation strategies.*

TASK 12 – Design Challenge: Selecting and Adapting a Learning Strategy | 8 Marks

Changed Requirement

The factory initially analyses 5,000 records with 4 sensor features using a logistic regression model trained by full-batch gradient descent. It must now process 2,000,000 records with 500 sensor features per component, predict the defect probability in near real time, and handle correlated features. Decide whether to retain, modify or redesign the strategy.

A. Impact Analysis | 2 Marks

a) Explain how the larger dataset and 500 features affect computation, memory and the training procedure.

Ans: Data size: 2,000,000 × 500 = 10^9 feature values, about 4 GB in float32 (8 GB in float64), compared with only 5,000 × 4 = 20,000 values before, i.e. 50,000 times more. **Computation:** one full-batch gradient costs O(m·d) = 10^9 multiply-adds and yields only one parameter update, so hundreds of epochs become very slow. **Memory:** the whole matrix may not fit in RAM/GPU memory, so data must be streamed in chunks. **Training procedure:** it must change to mini-batch (stochastic) updates with data loaders, a properly tuned learning rate and feature scaling; prediction must also be fast enough for near real-time use.

b) Discuss why dimensionality, feature correlation and gradient behaviour become important.

Ans: Dimensionality: in 500 dimensions the space is extremely sparse (curse of dimensionality) and the model has many more parameters to estimate, so the risk of overfitting and of noisy, irrelevant features rises.

Feature correlation: correlated sensors (e.g. temperature and pressure) make the Hessian X^TX ill-conditioned (multicollinearity): weight estimates become unstable with large variance, and the cost surface forms narrow elongated valleys in which gradient descent zig-zags and converges slowly.

Gradient behaviour: gradient magnitudes differ across features and (in deeper networks) can vanish or explode, and mini-batch gradients are noisy; so the choice of scaling, initialization, activation and optimizer (momentum/Adam) strongly affects convergence.

B. Evaluation of the Original Approach | 2 Marks

a) Evaluate whether a linear model trained with full-batch gradient descent is sufficient for the new requirement.

Ans: Not sufficient as it stands. The logistic model $\sigma(w^Tx + b)$ is cheap at inference (about 500 multiplications), so real-time prediction is fine, but (i) full-batch gradient descent is computationally impractical for 10^9 values (one update per full pass); (ii) a linear decision boundary may miss non-linear interactions among 500 correlated sensors; (iii) correlated features make training unstable. So the training procedure should at least be modified, and the model redesigned if the data are non-linear.

b) Explain the limitations of the original approach in terms of capacity, optimization cost and the curse of dimensionality.

Ans: Capacity: only 501 parameters and a linear boundary, giving high bias; it cannot capture complex non-linear defect patterns, even though 2 million samples could support a larger model.

Optimization cost: each update costs $O(m \cdot d) = 10^9$ operations, only 1 update per epoch, slow convergence because of ill-conditioning.

Curse of dimensionality: with 500 features the space is very sparse, irrelevant/redundant features add noise and variance, and there is no regularization or dimensionality reduction in the original design.

C. Adaptation or Redesign | 2 Marks

a) Propose a redesigned strategy using a Multilayer Perceptron trained with back-propagation and a suitable SGD variant.

Ans: Redesign (retain the probabilistic output, change the model and optimizer): a Multilayer Perceptron 500 → 128 (ReLU) → 64 (ReLU) → 1 (sigmoid), trained by back-propagation with **mini-batch SGD using Adam** (batch size 256).

Parameters: $(500 \cdot 128 + 128) + (128 \cdot 64 + 64) + (64 + 1) = 64{,}128 + 8{,}256 + 65 =$ **72,449**. Updates per epoch: 2,000,000 / 256 ≈ 7,813, compared with 1 for full batch. Inference needs one forward pass of about 72 thousand multiply-adds, which is fast enough for near real-time use. Logistic regression is the special case without hidden layers and stays as the baseline.

b) Explain how maximum likelihood (the loss function), regularization, dimensionality reduction and the learning rate can be incorporated.

Ans: Maximum likelihood (loss): treat each label as Bernoulli($\hat{y}$); the negative log-likelihood is the binary cross-entropy $L = -(1/m)\ \Sigma[y_i \ln \hat{y}_i + (1 - y_i) \ln(1 - \hat{y}_i)]$ (the same MLE principle as Task 3, with optional class weights for the rare defective class).

Regularization: L2 weight decay $\lambda\|W\|^2$ (equivalent to a Gaussian prior, MAP estimation), dropout and early stopping on a validation set, to keep the generalization gap small.

Dimensionality reduction: standardise features, then apply PCA (e.g. keep 95% of the variance) or feature selection / a bottleneck layer to decorrelate and compress the 500 sensors, which reduces parameters and the effect of multicollinearity.

Learning rate: Adam with $\eta \approx 10^{-3}$ and a decay or warm-up schedule, tuned on validation data; batch normalization and He initialization keep gradients well-scaled.

D. Evidence-Based Defence | 2 Marks

a) Propose experiments and evaluation measures to compare the redesigned approach with the original one.

Ans: Experiments: use a stratified train/validation/test split (time-ordered, to avoid leakage) and compare (1) original logistic regression with full-batch GD, (2) logistic regression with mini-batch SGD, (3) the proposed MLP + Adam, plus ablations (with/without PCA, L2, dropout; different hidden sizes). Repeat over several random seeds and report mean ± std.

Evaluation measures: precision, recall, F1, PR-AUC and ROC-AUC, confusion matrix (the defect class is rare), log-loss/calibration; training vs validation error and the generalization gap, learning curves, training time per epoch, memory use, convergence speed (loss vs time) and inference latency per component.

b) Justify the final design using mathematical reasoning, generalization, computational needs and data characteristics.

Ans: Final decision: redesign the model and modify the training procedure.

Mathematical reasoning: cross-entropy is the Bernoulli MLE, so the output stays a true defect probability; hidden ReLU layers add non-linearity (a linear model cannot capture interactions); Adam's per-parameter scaling and momentum counter ill-conditioning from correlated features.

Generalization: 2,000,000 samples against about 72,000 parameters (≈ 27 samples per parameter) is adequate, and L2/dropout/early stopping plus PCA control variance, so the gap should stay small.

Computation: mini-batch cost per update is $O(B \cdot P)$ independent of N, giving about 7,800 updates per epoch rather than 1, with streamed data that fit in memory; inference is a single fast forward pass.

Data characteristics: many correlated, differently scaled sensor features and a rare defect class, handled by standardization, PCA, class weights and precision/recall-based evaluation.

Evidence Expected: *Analysis of data scale and dimensionality, justified learning strategy, mathematical reasoning and a suitable evaluation plan.*

Final Mark Distribution – CO1

Task	Topics Covered	Marks
Task 1	Learning Algorithms – T, E, P and Paradigm	4
Task 2	Capacity, Overfitting and Generalization	3
Task 3	Maximum Likelihood Estimation – Bernoulli	4
Task 4	Maximum Likelihood Estimation – Gaussian and MSE	5
Task 5	Building a Machine Learning Algorithm	5
Task 6	Multilayer Perceptron – Forward Pass	4
Task 7	Back-propagation Algorithm	5
Task 8	Vanishing Gradient and Activation Functions	3
Task 9	Batch, Stochastic and Mini-batch Gradient Descent	3
Task 10	SGD Variants – Momentum, RMSProp, Adam	3
Task 11	Curse of Dimensionality	3
Sub Total		**42**
Task 12	Design Challenge – Selecting and Adapting a Learning Strategy	8

Task	Topics Covered	Marks
TOTAL		**50**